Array Mastery

9 patterns. ~59 problems. Pattern-first FAANG-prep cheatsheet.

@rathoreadiitya - Instagram

v1.0 · Updated 2026-05-22

Why this sheet exists

Arrays are the FIRST DSA topic every student touches and the LAST one they actually master. Most students "finish" arrays after 30 problems and still freeze on LC 42 or LC 239 in an interview. The fix isn't "more problems" - it's 9 patterns, ~59 problems, internalized cold. That's this sheet.

Nine patterns cover the full surface area of array problems: scan (prefix sums), squeeze (two pointers), window (sliding window), sum (Kadane), rearrange (in-place pointers), schedule (intervals), search (binary search), order (sorting toolkit - comparator, counting/bucket, merge sort, quickselect), and range-update (difference array - the inverse of prefix sums). Every array problem you'll hit in an interview reduces to one of these nine.

How to use: Crash course → 5 min (raw intuition). Patterns 1-9 in order - each has a warm-up, a walkthrough, traps, and 3-9 LC problems. Cross-pattern traps → read once before any interview. Revision drill → 25 minutes the night before.

Inside this sheet

Crash course (5 min)

An array is a contiguous block of memory with O(1) random access by index. That's it. Every "array problem" is really one of these 5 questions:

15. "What's the sum/product/min over this range?" → prefix sums (Pattern 1).
16. "Two indices that pair up to satisfy X." → two pointers (Pattern 2).
17. "Longest/shortest contiguous subarray with property X." → sliding window (Pattern 3).
18. "Maximum subarray sum / best buy-sell / contiguous best." → Kadane (Pattern 4).
19. "Rearrange in O(1) space" / "rotate" / "sort 0/1/2." → in-place pointers (Pattern 5).

Plus four structural patterns:

- Intervals (Pattern 6) → sort, then sweep.
- Binary search (Pattern 7) → sorted, OR a monotonic predicate over an answer range.
- Sorting toolkit (Pattern 8) → comparator / counting / merge sort / quickselect when default sort isn't enough.
- Difference array (Pattern 9) → many range UPDATES then a single materialise; the mirror of prefix sums.

If you can name which pattern fits in 10 seconds, you've solved 80% of array interviews. The rest is just code muscle.

Big insight you keep missing: binary search is not just "search in sorted array." It's SEARCH THE ANSWER. Whenever "smallest X such that P(X) is true" appears, BS over X. LC 875 (Koko) is the canonical example.

Pattern decision tree (problem → pattern in 10 seconds)

What the problem says	Pattern
"Sum / product over range [i, j]" or "subarray sum equals K"	1 - Prefix sums
"Pair / triplet that sums to X" or "container", "two sorted halves"	2 - Two pointers
"Longest / shortest contiguous subarray with X" or "sliding window of size k"	3 - Sliding window
"Maximum subarray", "max profit", "contiguous best"	4 - Kadane
"In-place rearrange", "rotate", "sort colors", "move zeroes"	5 - In-place pointers
"Meeting rooms", "merge intervals", "schedule", "overlap"	6 - Intervals
"Sorted array", "rotated sorted", "smallest k such that ...", "log n required"	7 - Binary search
"Sort by custom key", "top K frequent", "kth largest in O(N)", "sort linked list", "count inversions"	8 - Sorting toolkit
"Add val to every index in [l, r]" repeated K times, then "final array" / "capacity check" / "coverage check"	9 - Difference array

If two patterns seem to fit (e.g., LC 209 is sliding-window AND BS-on-answer), pick the one with simpler invariants for the interview - usually sliding window for monotone subarray problems, BS-on-answer for "minimum capacity / speed" framing.

The 9 patterns

Pattern 1 - Prefix sums & cumulative logic

One-liner: Precompute cumulative sums so any range query is O(1) - then layer a hashmap when you need "subarray with property X".

Mental model

Mental model: Prefix sum is the derivative-and-integral trick of arrays. P[i] = sum(nums[0..i-1]); then sum(nums[l..r]) = P[r+1] - P[l] in O(1). That's the entire vanilla version.

Hashmap upgrade: For LC 560 (sum equals K), at each r you want some l with P[r+1] - P[l] = K. Store every prefix sum seen in a counter; look up P[r+1] - K in O(1). Whole problem becomes O(n).

Invariant (vanilla): P[i] = nums[0] + ... + nums[i-1]; P[0] = 0 is the empty-prefix anchor.

Invariant (hashmap): counts[v] = number of indices l with P[l] == v. Seed with {0: 1} so subarrays starting at index 0 count.

Spot signals

- "Sum over range [i, j]" → vanilla prefix sum.
- "Subarray sum equals K / divisible by K / mod K" → prefix + hashmap.
- "Product of array except self" → prefix and suffix products.
- "Continuous subarray sum that's a multiple of K" → store P % k in the hashmap.

Walkthrough - LC 560 Subarray Sum Equals K

The canonical prefix + hashmap. Seed the counter with {0: 1}, then iterate.

Python

```
def subarraySum(nums, k):
    from collections import defaultdict
    counts = defaultdict(int)
    counts[0] = 1            # seed: empty prefix has sum 0
    pref = 0
    ans = 0
    for x in nums:
```

```
        pref += x
        ans += counts[pref - k]   # how many earlier prefixes give a window
summing to k?
        counts[pref] += 1
    return ans
```

Common traps

- Forgetting the {0: 1} seed. Without it, subarrays starting at index 0 don't count → silent off-by-one. This is the #1 LC 560 bug.
- Updating the hashmap BEFORE the lookup. You'd count the zero-length subarray ending at r. Order: lookup, then insert.
- Confusing "subarray" (contiguous) with "subsequence" (any subset). Prefix sums only work for contiguous.
- Integer overflow in C++/Java on long prefix sums - use long long / long. Python is safe.

Warm-up first - build the mechanics

Do this cold before LC 560. It locks in the "build the prefix array, then query in O(1)" loop you'll layer the hashmap trick on top of.

#	Problem	Difficulty	Link
303	Range Sum Query Immutable - precompute prefix array, return P[j+1] - P[i]	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	560	Subarray Sum Equals K - the canonical prefix + hashmap; if this isn't reflex, no other prefix problem will be	Medium	LeetCode
2	238	Product of Array Except Self - two-pass prefix/suffix, no division, O(1) extra space; iconic Amazon ask	Medium	LeetCode
3	523	Continuous Subarray Sum - the P % k variant; same template, mod instead of raw sum	Medium	LeetCode
4	724	Find Pivot Index - prefix vs suffix sum equality; the simplest LC 150 prefix-sum problem	Easy	LeetCode
5	304	Range Sum Query 2D Immutable - the 2D prefix-sum upgrade; precompute once, O(1) per query	Medium	LeetCode
6	169	Majority Element - Boyer-Moore voting; single-pass cumulative count, O(1) space; mandatory LC 150	Easy	LeetCode

Step	#	Problem	Difficulty	Link
7	217	Contains Duplicate - cumulative seen-set; the simplest hashing-on-array; the LC 150 floor	Easy	LeetCode
8	128	Longest Consecutive Sequence - hashset of nums; only start counting when num-1 is missing; O(N) bar-raiser	Medium	LeetCode

Pattern 2 - Two pointers on arrays

One-liner: When the array is sorted (or you're free to sort it), put one pointer at each end and CONVERGE based on what the comparison tells you.

Mental model

Mental model: Two pointers turns $O(n^2)$ brute force into O(n) by discarding half the candidate pairs each step.

Opposite ends (l=0, r=n-1): each step move the pointer whose value loses the comparison. Works when the comparison is monotone in the moved pointer.

Same direction (slow=0, fast=0): fast reads, slow writes - overwrite in place. Works for compact / move zeroes / dedup.

Aha for opposite-ends: if nums[l] + nums[r] < target, every pair using l and any index < r is also too small (sorted!). Never revisit - move l up. Symmetric for >.

Invariant (opposite): all pairs (i, j) with i < l or j > r have been examined and ruled out.

Invariant (same-dir): nums[0..slow-1] is the finished output region; nums[fast..] is unexamined.

Spot signals

- "Sorted array" / "pair / triplet sums to X"
- "Container with most water" / "trapping rain water"
- "Remove duplicates / move zeroes in place"
- "Closest pair / merge two sorted arrays"

Walkthrough - LC 11 Container With Most Water

Move the SHORTER side every iteration - it's the only chance to find a taller bounding wall.

Python

```
def maxArea(height):
    l, r = 0, len(height) - 1
```

```
    best = 0
    while l < r:
        h = min(height[l], height[r])
        best = max(best, h * (r - l))
        if height[l] < height[r]:
            l += 1
        else:
            r -= 1
    return best
```

Common traps

- Moving the wrong pointer. In LC 11, move the SHORTER side - moving the taller one can never improve area (width shrinks, height capped by shorter anyway). Beginners default to "always move l" and fail.
- Not deduping in 3Sum. After fixing i, when you advance l and r, skip equal neighbors or you'll emit duplicate triplets.
- Off-by-one on `while l < r` vs `l <= r`. For pair problems (LC 11, LC 167), use l < r (you need two distinct indices).
- Sorting LC 1 (Two Sum) to use two pointers. LC 1 wants the ORIGINAL indices - sorting destroys them. Use a hashmap instead.

Warm-up first - build the mechanics

Do this cold before LC 11. It teaches "use a hashmap to remember what you've seen" - the seed of the same-direction-pointer reflex.

#	Problem	Difficulty	Link
1	Two Sum - one-pass hash: for each x, look up target - x	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	11	Container With Most Water - the cleanest "move the shorter side" argument; bar-raiser for 2P intuition	Medium	LeetCode
2	15	3Sum - fix-one + two-pointer + dedup; the most-asked array problem on the planet	Medium	LeetCode
3	42	Trapping Rain Water - the boss; two-pointer with running maxL/maxR, O(n) time O(1) space; mandatory bar-raiser	Hard	LeetCode

Step	#	Problem	Difficulty	Link
4	125	Valid Palindrome - the canonical skip-and-compare two-pointer; the warm-up for every other 2P problem	Easy	LeetCode
5	31	Next Permutation - find pivot from the right, swap with the smallest larger suffix element, reverse the tail; classic 2P trick	Medium	LeetCode

Pattern 3 - Sliding window (fixed + variable + monotonic deque)

One-liner: The window [l, r] is your CURRENT subarray; expand r to grow, contract l to fix violations, track the best as you go.

Mental model

Mental model: Sliding window is two pointers + a running summary (sum, count, freq map, deque). Three flavors:

Variable window: `for r: while bad: l++` - for "longest with property X" / "shortest with constraint Y".

Fixed window (size k): slide a window of fixed width across; add nums[r], drop nums[r-k].

Monotonic deque: store INDICES in a deque whose values are strictly decreasing → front = window max. For LC 239.

Invariant (variable): at every iteration, [l, r] is the longest valid window ending at r (or shortest for "minimum" - invert the contract direction).

Invariant (deque): deque[0] is the index of the max in the current window; values strictly decreasing front-to-back.

Spot signals

- "Longest / shortest subarray or substring with property X"
- "Window of size k" / "sliding maximum / minimum"
- "At most K distinct" / "containing all chars of T"
- "Maximum sum of k consecutive elements"

Walkthrough - LC 209 Minimum Size Subarray Sum

Variable window, "shortest with constraint". Expand r, shrink l while constraint still holds.

Python

```python
def minSubArrayLen(target, nums):
    l = 0
    s = 0
    best = float('inf')
    for r, x in enumerate(nums):
        s += x
        while s >= target:                # constraint met -> try to shrink
            best = min(best, r - l + 1)
            s -= nums[l]
            l += 1
    return best if best != float('inf') else 0
```

Walkthrough - LC 239 Sliding Window Maximum (the boss)

Monotonic deque of INDICES. Front = max. Kick out smaller-or-equal from the back when pushing; drop front when it falls out of the window.

Python

```python
from collections import deque
def maxSlidingWindow(nums, k):
    dq = deque()  # indices; values nums[dq] strictly decreasing
    out = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] < x:    # kick out smaller from the back
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:                # front out of window -> discard
            dq.popleft()
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out
```

Common traps

- Shrinking in the wrong direction. "Longest with at most K distinct" shrinks when > K (violated). "Shortest with sum ≥ target" shrinks when ≥ target (satisfied). Read the problem twice.
- Off-by-one on window length. r - l + 1 (inclusive). Beginners write r - l and undercount by 1.
- Storing values instead of indices in the deque. You'll fail to drop the front when the window slides past it.
- Not contracting in a `while`. A single `if` can leave the window invalid for more than one step.

Warm-up first - build the mechanics

Do this cold before LC 209. It's the simplest "expand r, contract l when constraint breaks" loop - same skeleton, no edge cases.

#	Problem	Difficulty	Link
643	Maximum Average Subarray I - fixed window of size k; running sum	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	209	Minimum Size Subarray Sum - variable window, "shortest with constraint" template; the mirror of LC 3	Medium	LeetCode
2	3	Longest Substring Without Repeating Characters - variable window + freq map; the most-asked sliding window	Medium	LeetCode
3	239	Sliding Window Maximum - the boss; monotonic deque. Own this and every window max/min variant follows	Hard	LeetCode
4	904	Fruit Into Baskets - variable window with at most 2 distinct values; the cleanest "K distinct" warm-up	Medium	LeetCode

Want the substring deep dive? See the Two-Pointers + Hashing sheet for LC 76, 424, 438, 567 - fixed-length + freq-map variants live there.

Pattern 4 - Kadane & max-subarray family

One-liner: At every index, the best subarray ending HERE is either "extend the previous best" or "start fresh from here" - pick the larger.

Mental model

Mental model: Kadane is THE O(n) DP-in-disguise. Define curr = max sum of any subarray ENDING at index i. One-line recurrence: curr = max(nums[i], curr + nums[i]); best = max(best, curr).

State picking: LC 121 (stock): track min-so-far instead of curr-sum. LC 152 (product): track BOTH max and min (negative × negative = big positive). LC 918 (circular): use (total - minKadane) for the wrap-around case.

Invariant: after processing index i, curr = best subarray sum ending at i; best = overall best in nums[0..i].

Spot signals

- "Maximum subarray sum / product / profit"
- "Contiguous" + "best / largest / max"
- "Stock with one buy + one sell"
- "Circular array max sum"

Walkthrough - LC 53 Maximum Subarray (canonical)

Two variables, one pass. If you can't write this in 30 seconds from memory, you don't own the pattern.

Python

```
def maxSubArray(nums):
    curr = best = nums[0]
    for x in nums[1:]:
        curr = max(x, curr + x)           # start fresh OR extend
        best = max(best, curr)
    return best
```

Walkthrough - LC 152 Maximum Product Subarray (the trap)

Track BOTH max and min - a large negative product becomes a huge positive after one more negative.

Python

```
def maxProduct(nums):
    curr_max = curr_min = best = nums[0]
    for x in nums[1:]:
        if x < 0:
            curr_max, curr_min = curr_min, curr_max  # swap: x negative flips
        curr_max = max(x, curr_max * x)
        curr_min = min(x, curr_min * x)
        best = max(best, curr_max)
    return best
```

Common traps

- Initializing best = 0. If all numbers are negative, the answer is the largest single negative - not 0. Initialize best = nums[0].

- Not tracking the min in product Kadane. A large negative product becomes a huge positive after one more negative - you MUST track both extremes.
- All-negative edge in LC 918 (circular). The (total - minKadane) trick gives 0 if every element is negative. Handle separately: return regular Kadane in that case.
- Confusing buy-sell stock for Kadane. LC 121 is max(price - min_so_far) - a prefix-min problem, not classic Kadane (though equivalent in disguise).

Warm-up first - build the mechanics

Do this cold before LC 53. It frames "track the min so far, take the best gain" - Kadane in disguise, simpler invariants.

#	Problem	Difficulty	Link
121	Best Time to Buy and Sell Stock - track min_so_far, update best = max(best, price - min_so_far)	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	53	Maximum Subarray - the canonical Kadane; 30-second write from memory or you don't own the pattern	Medium	LeetCode
2	152	Maximum Product Subarray - the track-both-extremes upgrade; negative-number reflex	Medium	LeetCode
3	918	Maximum Sum Circular Subarray - the "regular OR (total - minKadane)" trick; bar-raiser at MSFT / AMZN	Medium	LeetCode

Pattern 5 - In-place rearrangement (Dutch flag, rotate, move zeroes)

One-liner: Rearrange the array with O(1) extra space using 2 or 3 pointers - write region + read region + (sometimes) unknown region.

Mental model

Mental model: The skeleton is partition by pointers.

Two pointers (write/read): slow = next write position; fast scans forward. For "move zeroes," "remove duplicates," "compact valid elements."

Three pointers (Dutch flag): low (next 0 slot), mid (cursor), high (next 2 slot from back). For "sort colors / 0-1-2 sort." The mid pointer drives.

Reverse-reverse-reverse: for "rotate by k": reverse whole array, reverse first k, reverse last n-k. Three O(n) passes = O(n) time, O(1) space.

Invariant (Dutch flag): nums[0..low-1] all 0s; nums[low..mid-1] all 1s; nums[high+1..n-1] all 2s; nums[mid..high] unprocessed.

Spot signals

- "In-place / O(1) extra space"
- "Move zeroes" / "remove duplicates"
- "Sort colors / 0s 1s 2s"
- "Rotate array by k"
- "Cyclic sort / find missing or duplicate in 1..n"

Walkthrough - LC 75 Sort Colors (Dutch flag)

On value 2, swap and DO NOT advance mid - the value swapped in is unknown, must be re-examined.

Python

```
def sortColors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:  # nums[mid] == 2
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
            # DO NOT advance mid - swapped-in value is unknown
```

Common traps

- Advancing mid after the 2-swap in Dutch flag. The value swapped in from high is unprocessed - re-examine it. This is the #1 LC 75 bug.
- Off-by-one on k % n for rotation. k can be > n; always do k = k % n first or you'll over-rotate.

- Treating "find duplicate" (LC 287) as sort-then-scan. Sorting destroys the O(1) space guarantee. Use Floyd's cycle detection (treat array as a linked list of indices).
- Returning a new array when the problem says "in place." Reads as "you don't know what in-place means" - instant interview red flag.

Warm-up first - build the mechanics

Do this cold before LC 75. It locks in the "slow writes, fast reads" two-pointer rhythm - Dutch flag is the same idea with one more pointer.

#	Problem	Difficulty	Link
283	Move Zeroes - same-direction two-pointer; swap non-zero to slow, advance both	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	75	Sort Colors - the Dutch flag classic; bar-raiser for in-place fluency	Medium	LeetCode
2	189	Rotate Array - the reverse-reverse-reverse trick; only good answer at O(1) extra space	Medium	LeetCode
3	287	Find the Duplicate Number - Floyd cycle detection on an array; mind-bending the first time, iconic afterwards	Medium	LeetCode
4	41	First Missing Positive - cyclic sort: place each value v at index v-1, then scan; O(N) time, O(1) space; iconic FAANG hard	Hard	LeetCode
5	448	Find All Numbers Disappeared in an Array - mark visited via negation OR cyclic sort; the cleanest cyclic-sort warm-up	Easy	LeetCode
6	48	Rotate Image - transpose then reverse each row; the canonical in-place matrix rotation	Medium	LeetCode
7	54	Spiral Matrix - walk by layers, contracting bounds; classic matrix simulation, painful edge cases	Medium	LeetCode
8	73	Set Matrix Zeroes - use first row/col as in-place markers; O(1) extra space; Microsoft favourite	Medium	LeetCode
9	66	Plus One - right-to-left carry simulation; the simplest array-as-bignum exercise	Easy	LeetCode

Pattern 6 - Interval merging & overlaps

One-liner: Sort intervals by start (for merging) or by end (for max non-overlapping), then sweep left-to-right with a greedy choice.

Mental model

Mental model: Intervals = sort + sweep. The key decision is WHAT to sort by.

Sort by start: use for merging or inserting. After sort, two intervals overlap iff prev.end >= curr.start - extend prev.end = max(prev.end, curr.end).

Sort by end: use for max non-overlapping (activity-selection / LC 435). The earlier an interval ENDS, the more room it leaves.

Invariant (merge): after the i-th sorted interval, merged contains all non-overlapping merged intervals from intervals[0..i], with merged[-1] possibly still extendable.

Spot signals

- "Merge intervals" / "insert interval"
- "Meeting rooms" / "minimum number of conference rooms"
- "Non-overlapping intervals" / "max meetings you can attend"
- "Schedule" / "overlap"

Walkthrough - LC 56 Merge Intervals

Sort by start, walk with a running prev; overlap → extend, disjoint → append.

Python

```python
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= merged[-1][1]:                  # overlap -> extend
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])               # disjoint -> append
    return merged
```

Walkthrough - LC 435 Non-overlapping Intervals (sort by END)

Activity-selection greedy: keep intervals whose start ≥ last_end. Min removals = total - max kept.

Python

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])          # sort by END
    kept = 0
    last_end = float('-inf')
    for s, e in intervals:
        if s >= last_end:                       # no overlap -> keep
            kept += 1
            last_end = e
    return len(intervals) - kept
```

Common traps

- Sorting by start when you needed end (or vice versa). Single biggest interval-problem bug. Decide upfront: merging = start, max-non-overlapping = end.
- Mishandling identical endpoints. Decide whether [1,2] and [2,3] overlap (touch = overlap? or not?). The problem statement defines it - read carefully.
- Forgetting to handle the last interval. When sweeping with a running prev, append prev to the output after the loop. Easy to miss.
- Mutating the input. Some interviewers ding you for sorting in place. Make a copy if instructed.

Warm-up first - build the mechanics

Do this cold before LC 56. It locks in "sort by start, then walk with a running prev" without the merge-extension step on top.

#	Problem	Difficulty	Link
228	Summary Ranges - single pass; detect when consecutive integers break; emit a range string per run	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	56	Merge Intervals - the canonical merge; mandatory FAANG problem	Medium	LeetCode
2	57	Insert Interval - the skip / merge / append three-zone walk; asked at Google	Medium	LeetCode
3	435	Non-overlapping Intervals - the sort-by-END greedy; activity-selection classic in interview disguise	Medium	LeetCode

Step	#	Problem	Difficulty	Link
4	986	Interval List Intersections - two-pointer walk over two pre-sorted interval lists; computes pairwise overlaps in O(N+M)	Medium	LeetCode

Pattern 7 - Binary search on arrays

One-liner: If the data is sorted OR the answer space is monotonic, binary-search it - lo, hi, mid, halve the search space each iteration.

Mental model

Mental model: Binary search is one of two things - (1) search a value in a sorted array; (2) search the ANSWER over a monotonic predicate.

The unlock: "Smallest X such that P(X) is true" where P is monotone (false, false, false, true, true, true) → BS over X. Turns junior into senior. LC 875 Koko is the canonical example.

Invariant: at every iteration, the answer is guaranteed to lie in [lo, hi]. Each iteration shrinks the range. Pick boundary updates that PRESERVE this - that's the entire correctness argument.

Three templates: (a) find exact target: `lo <= hi`, `hi = mid - 1`, return mid or -1; (b) lower-bound: `lo < hi`, `hi = mid` on true / `lo = mid + 1` on false, return lo; (c) rotated half-pick: `lo <= hi` with branch on which half is sorted.

Spot signals

- "Sorted array" / "rotated sorted array"
- "Find first / last position of"
- "Smallest k such that X" / "minimum capacity / speed to ..."
- "O(log n) required" / "less than O(n)"
- "Find peak / find pivot"

Walkthrough - LC 704 Binary Search (the template)

The classic. lo + (hi - lo) // 2 is overflow-safe in C++/Java; Python ints are unbounded.

Python

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2     # overflow-safe in C++/Java
```

```
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Walkthrough - LC 875 Koko Eating Bananas (BS on the answer)

Predicate P(k) = "can she finish at speed k?" is monotone (faster → fewer hours). BS k over [1, max(piles)].

Python

```
import math
def minEatingSpeed(piles, h):
    def can_finish(k):
        return sum(math.ceil(p / k) for p in piles) <= h

    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_finish(mid):
            hi = mid                    # mid works - try smaller
        else:
            lo = mid + 1                # mid too slow - go faster
    return lo                            # lo == hi == answer
```

Common traps

- (lo + hi) // 2 overflow in C++/Java. Use lo + (hi - lo) // 2. Python ints are unbounded - safe.
- Off-by-one on `lo <= hi` vs `lo < hi`. With `lo <= hi` update `hi = mid - 1`. With `lo < hi` update `hi = mid`. Mismatching → infinite loop.
- Wrong half pruned in rotated array (LC 33). First check WHICH half is sorted (nums[lo] <= nums[mid] → left sorted), THEN check if target lies in that sorted half before pruning.
- Forgetting the monotone predicate is a HARD requirement for BS-on-answer. If P(X) flips back and forth, you can't binary search - go back to brute / sliding window.
- Returning lo vs hi ambiguity. With the `lo < hi` template, lo == hi at termination - return either. Pick one and stay consistent.

Warm-up first - build the mechanics

Do this cold before LC 704 even. It's the "world's simplest BS" - purely to internalize the lo, hi, mid, halve-the-range loop.

#	Problem	Difficulty	Link
35	Search Insert Position - find target OR insert index; standard `lo <= hi` template, return lo	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	704	Binary Search - the canonical template; write from memory	Easy	LeetCode
2	34	Find First and Last Position of Element in Sorted Array - lower-bound + upper-bound twin search	Medium	LeetCode
3	33	Search in Rotated Sorted Array - decide-which-half-is-sorted variant; iconic FAANG screen	Medium	LeetCode
4	153	Find Minimum in Rotated Sorted Array - pivot search; compare mid to hi (not lo)	Medium	LeetCode
5	875	Koko Eating Bananas - the boss: BS on the ANSWER. Every "min capacity / speed" problem becomes 5 lines	Medium	LeetCode
6	162	Find Peak Element - BS on an unsorted array using the slope; mid vs mid+1 decides which half holds a peak	Medium	LeetCode
7	658	Find K Closest Elements - BS for the window start; compares arr[mid] vs arr[mid+k] to slide the window	Medium	LeetCode

Pattern 8 - Sorting toolkit (custom comparator + counting/bucket + merge sort + quickselect)

One-liner: Sorting is a TOOL, not a default. Reach for `sort + key=lambda` first; upgrade to counting/bucket for bounded values, merge sort for stability + inversion counting, quickselect for kth-element in O(N) avg.

Mental model

Mental model: Default = Python's Timsort - O(N log N), stable, in-place enough. Reach for a specialised sort only when (a) the value range is bounded → counting/bucket O(N); (b) you need stability + cross-half counting → merge sort; (c) you only need the kth element → quickselect O(N) average.

Custom comparator: `sorted(arr, key=lambda x: ...)` for derived keys (multi-field tuples allowed). Use

`functools.cmp_to_key(lambda a, b: ...)` only when the rule is RELATIONAL (e.g. LC 179 “ab vs ba”).

Counting / bucket sort: Values in [0, K] → allocate count[K+1], one pass to fill, one to rebuild. O(N + K). Frequency-as-bucket-index gives LC 347 in O(N) (beats heap).

Merge sort: Recursive split + 2-pointer merge. O(N log N) guaranteed (no quicksort worst-case). Stable. The merge step ALSO counts inversions (LC 315 / 493) and sorts linked lists in O(1) space (LC 148).

Quickselect: Lomuto partition with RANDOM pivot. After partition, pivot is at sorted position p. If p == k return; else recurse one side. Average O(N), worst $O(N^2)$ (random pivot mitigates). Bar-raiser alternative to heap-of-size-K for LC 215.

Invariant (quickselect): After each partition, the kth element is confined to ONE shrinking side. Iterate → logarithmic depth, linear partition work → O(N) average.

Spot signals

- “Sort by length / frequency / composite key” → custom comparator
- “Sort N elements with values in [0, K]” / “top K frequent” → counting / bucket sort
- “Sort linked list O(1) space” / “count inversions / smaller after self” → merge sort
- “Kth largest / kth closest / k closest points” + “O(N) average” hint → quickselect
- “Don't use built-in sort” / “implement N log N sort from scratch” → merge sort (always preferred over manual quicksort)

Walkthrough - LC 179 Largest Number (custom comparator via cmp_to_key)

Pairwise compare a + b vs b + a. The greater concatenation comes first.

Python

```python
from functools import cmp_to_key

def largestNumber(nums):
    nums = [str(n) for n in nums]
    nums.sort(key=cmp_to_key(lambda a, b: 1 if a + b < b + a else -1))
    return '0' if nums[0] == '0' else ''.join(nums)
```

Walkthrough - LC 347 Top K Frequent (bucket sort by frequency)

Bucket index = frequency. Walk descending; collect top K. O(N) beats the heap solution.

Python

```python
from collections import Counter

def topKFrequent(nums, k):
    cnt = Counter(nums)
```

```
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in cnt.items():
        buckets[freq].append(num)
    res = []
    for i in range(len(buckets) - 1, -1, -1):
        for num in buckets[i]:
            res.append(num)
            if len(res) == k: return res
    return res
```

Walkthrough - LC 148 Sort List (merge sort on linked list)

Slow/fast to split; recurse; merge with dummy head. O(N log N) time, O(1) space.

Python

```
def sortList(head):
    if not head or not head.next: return head
    slow, fast, prev = head, head, None
    while fast and fast.next:
        prev = slow
        slow = slow.next
        fast = fast.next.next
    prev.next = None                                  # split
    left  = sortList(head)
    right = sortList(slow)
    return merge(left, right)

def merge(a, b):
    dummy = tail = ListNode(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

Walkthrough - LC 215 Kth Largest (quickselect, random pivot)

Lomuto partition, random pivot to dodge sorted-input O(N^2). Recurse into the side containing k.

Python

```
import random
```

```
def findKthLargest(nums, k):
    k = len(nums) - k                                # convert to kth smallest
index
    def partition(l, r):
        p = random.randint(l, r)
        nums[p], nums[r] = nums[r], nums[p]
        pivot, store = nums[r], l
        for i in range(l, r):
            if nums[i] < pivot:
                nums[i], nums[store] = nums[store], nums[i]
                store += 1
        nums[store], nums[r] = nums[r], nums[store]
        return store
    l, r = 0, len(nums) - 1
    while True:
        p = partition(l, r)
        if p == k: return nums[p]
        if p < k: l = p + 1
        else: r = p - 1
```

Common traps

- Defaulting to sort when hash / heap / quickselect is faster. Always estimate complexity in O(N), O(N log K), O(N log N) before coding.
- Using `key=lambda` when the rule is RELATIONAL. Some rules can't be reduced to a single key - use `cmp_to_key`.
- LC 179 forgetting the '0' edge case. If all numbers are 0, return '0' not '000'.
- Counting sort with huge K (e.g. 10^9). O(K) space prohibitive; use comparison sort or hash-based bucket.
- Quickselect WITHOUT random pivot. Sorted input → O(N²) → TLE on adversarial tests. Always shuffle / random.
- LC 215 mixing up kth largest vs kth smallest. Convert via `k = len(nums) - k` for the kth-smallest implementation.
- Sorting in LC 287 (Find Duplicate) when the constraint is O(1) space. Floyd's cycle detection is the only correct answer there - see Pattern 5.
- Trying to count inversions OUTSIDE the merge step. The MERGE is where right-side elements jump ahead of left-side - count there.

Warm-up first - build the mechanics

Solve LC 1356 cold before LC 179. Sort integers by `(bin(x).count('1'), x)` - the simplest custom key - locks the key-as-tuple idiom in 3 minutes.

#	Problem	Difficulty	Link

#	Problem	Difficulty	Link
1356	Sort Integers by The Number of 1 Bits - multi-key sort warm-up	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	179	Largest Number - cmp_to_key with pairwise concat; mandatory FAANG comparator problem	Medium	LeetCode
2	937	Reorder Data in Log Files - multi-key sort separating letter and digit logs; common at Amazon	Medium	LeetCode
3	347	Top K Frequent Elements - bucket sort by frequency; O(N) beats the heap solution	Medium	LeetCode
4	148	Sort List - merge sort on a linked list; O(N log N) time, O(1) space; mandatory FAANG	Medium	LeetCode
5	912	Sort an Array - implement any O(N log N) sort; great for practising merge sort from scratch	Medium	LeetCode
6	215	Kth Largest Element in an Array - the canonical Quickselect; bar-raiser at FAANG	Medium	LeetCode
7	973	K Closest Points to Origin - Quickselect on distance; classic Amazon screen	Medium	LeetCode

When sorting feels wrong, ask: HASH (LC 1 Two Sum), HEAP (top-K streaming), FLOYD'S (LC 287 duplicate in O(1) space), or MONOTONIC STACK (next-greater-element). Sort is the FALLBACK - not the default.

Pattern 9 - Difference array (range updates, the inverse of prefix sums)

One-liner: When you need to apply MANY range updates and then read the final array, mark only the endpoints (diff[l] += val, diff[r+1] -= val) and do ONE prefix-sum pass at the end. K updates in O(K + N) instead of O(K × N).

Mental model

Mental model: Prefix sum answers "sum over range" queries in O(1) AFTER an O(N) build. Difference array is the mirror: it applies "add val to every index in range" updates in O(1) per update, then a single O(N) build materialises the final array. Same trick, opposite direction.

The +/- endpoint trick: For an update "add val to indices [l, r] (inclusive)", write `diff[l] += val` and `diff[r+1] -= val`. After all K updates, prefix-sum `diff` once - the value at index i is the sum of all val contributions that cover i.

Why r+1, not r: The decrement cancels the increment at the FIRST index that is NO LONGER in range. r+1 is half-open boundary. Off-by-one here is the single biggest bug in this pattern - size the diff array as N+1 to give yourself room.

Two flavours: (a) Materialise final array (LC 1109) - prefix-sum at the end and return. (b) Check a running constraint (LC 1094) - prefix-sum on the fly and bail the moment any cumulative value exceeds the cap. Same diff array, different read pattern.

Invariant: After all K updates and one prefix-sum pass, `diff[i] = sum of val for every update whose range covers index i`. Holds even when ranges overlap arbitrarily.

Spot signals

- "Apply K range updates (add val from index l to r) then return the final array" → difference array
- "Can we serve every trip / booking / shift given capacity X" → diff array + running sum + bail on overflow
- "For each query [l, r], increment range" AND "queries are batched (you see them all upfront)" → diff array beats segment tree
- K updates with N indices and K × N would TLE → the O(K + N) version is the diff-array trick

Walkthrough - LC 1109 Corporate Flight Bookings (canonical materialise pattern)

Bookings are [first, last, seats] using 1-indexed flight numbers. Naively applying each booking would be O(K × N). With diff array: mark each booking at first-1 (+seats) and last (-seats), then prefix-sum once.

Python

```
def corpFlightBookings(bookings, n):
    diff = [0] * (n + 1)                              # extra slot for the -= at
r+1
    for first, last, seats in bookings:
        diff[first - 1] += seats                      # 1-indexed -> 0-indexed
start
        diff[last] -= seats                           # last is already (r+1) in
0-indexed
    for i in range(1, n):
        diff[i] += diff[i - 1]                        # prefix-sum in place
    return diff[:n]
```

Walkthrough - LC 1094 Car Pooling (capacity-check variant, no need to materialise)

Each trip [numPassengers, from, to] picks up at `from` and drops at `to`. The car has fixed capacity. Use diff array on positions 0..1000 (problem cap), prefix-sum on the fly, return False the moment running sum exceeds capacity.

Python

```
def carPooling(trips, capacity):
    diff = [0] * 1001                                   # positions 0..1000
(problem constraint)
    for num, start, end in trips:
        diff[start] += num                              # board at start
        diff[end]   -= num                              # drop at end (end is
already r+1)
    curr = 0
    for d in diff:
        curr += d                                       # rolling prefix sum
        if curr > capacity: return False
    return True
```

Common traps

- Putting -= val at index r instead of r+1. This shrinks the range by one element. Always size the diff array as N+1 so r+1 is in bounds.
- Mixing 1-indexed and 0-indexed. LC 1109 uses 1-indexed flight numbers; the conversion is `diff[first - 1] += val; diff[last] -= val` because `last` in 1-indexed equals `r+1` in 0-indexed.
- Forgetting to prefix-sum at the end (in the materialise variant). The diff array alone is meaningless - only after the pass does each index hold the correct value.
- Using a diff array when there is ONE query interleaved with each update. Diff array shines for BATCHED updates; per-query mix needs a segment tree or BIT.
- Picking diff array for a 2D range update without lifting to the 2D variant. The 2D version exists (mark 4 corners with +/+/-/-) but the 1D pattern doesn't generalise by accident - know which one the problem needs.

Warm-up first - build the mechanics

Solve LC 1893 cold before LC 1109. It's the boolean version (cover-check, not value-accumulation) and locks in the +/- endpoint instinct in 5 minutes.

#	Problem	Difficulty	Link
1893	Check if All the Integers in a Range Are Covered - mark +1 at start, -1 at end+1, prefix-sum, verify all positions in [left, right] are > 0	Easy	LeetCode

Practice

Solve in this order - each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	1109	Corporate Flight Bookings - THE canonical 1D diff array; materialise final array in O(N + K)	Medium	LeetCode
2	1094	Car Pooling - diff array + on-the-fly prefix sum with early bail; capacity-check variant	Medium	LeetCode
3	2381	Shifting Letters II - string version: each shift is a range update on character offsets; modular arithmetic + diff array	Medium	LeetCode

Cross-pattern traps (read once before any interview)

Trap	Pattern most at risk	How to dodge
Forgetting {0: 1} seed in prefix-count hashmap	1	Write the seed FIRST line after the hashmap declaration. Muscle memory.
Moving the wrong pointer in two-pointer convergence	2	Say out loud: "I move the side whose value can be IMPROVED by moving."
Off-by-one on r - l + 1 window length	3	Always inclusive: r - l + 1. Drill 5 problems with deliberate length checks.
Initializing best = 0 in Kadane on all-negative arrays	4	Initialize best = nums[0]. Always.
Advancing mid after a 2-swap in Dutch flag	5	Only advance mid on values 0 and 1. On value 2, swap and STAY (re-examine mid).
Sorting intervals by the WRONG end	6	Merging → sort by start. Max non-overlapping → sort by END. Drill the dual.
Off-by-one on `lo <= hi` vs `lo < hi` in BS	7	Pair loop with boundary update - `<= hi` ⇒ `hi = mid - 1`. `< hi` ⇒ `hi = mid`. Never mix.
Defaulting to sort when hash / heap / quickselect is faster	8	Estimate O(N), O(N log K), O(N log N) BEFORE coding. Sort is the fallback, not the default.
Quickselect without random pivot	8	Sorted input → $O(N^2)$ → TLE. Always `random.randint(l, r)` + swap to end before Lomuto.
Off-by-one on diff[end+1] -= val (using end instead of end+1)	9	Diff array length is N+1 for half-open [start, end+1). If the range is inclusive [l, r], the decrement goes at index r+1, NOT r.

Trap	Pattern most at risk	How to dodge
Integer overflow on cumulative sum / product (C++/Java)	1, 4	Use long long / long. Python is safe; switching to a stricter language in interview is the common surprise.

Pre-interview revision drill (25 minutes)

Run this the night before:

20. 30 seconds per pattern, name the invariant out loud (you should be able to do all 9).
21. 3 minutes per pattern, write the skeleton code from memory (no IDE, paper or a comment).
22. 5 minutes, walk through LC 42 (Trapping Rain Water) two-pointer logic - if you can argue why moving the shorter side is correct, your two-pointer intuition is solid.
23. 5 minutes, walk through LC 875 (Koko) - if you can name the predicate P(k) and argue monotonicity, your BS-on-answer is solid.
24. 3 minutes, write LC 215 (Quickselect partition) from memory - the random-pivot Lomuto template is the sorting bar-raiser.
25. 2 minutes, name the #1 trap in each pattern (the cross-pattern table above).

If you fumble any of these → don't open more new problems, go re-do the warm-up + canonical problem for that pattern.

8-day study plan

Day	Focus	Problems	Notes
1	Pattern 1 + Pattern 9	LC 303, 560, 238, 1893, 1109	Prefix sums + difference array together - the same trick run forward (query) vs in reverse (update).
2	Pattern 2 + Pattern 3 start	LC 1, 11, 15, 42	Two pointers all-in. LC 42 deserves its own session - argue the invariant out loud.
3	Pattern 3 boss + Pattern 4	LC 643, 209, 3, 239, 53, 152	LC 239 (monotonic deque) is the hardest piece - budget extra time. Kadane next.
4	Pattern 4 boss + Pattern 5	LC 918, 121, 283, 75, 189	Rotate sneakier than it looks - write all three versions (extra array, cyclic, reverse-3x).
5	Pattern 5 boss + Pattern 6	LC 287, 228, 56, 57	LC 287 (Floyd) is the "wow" problem - solve, sleep, re-solve next morning.
6	Pattern 6 boss + Pattern 7	LC 435, 35, 704, 34, 33	Binary search day - write the three templates from memory before opening LC.

Day	Focus	Problems	Notes
7	Pattern 7 finish + Pattern 8 start	LC 153, 875, 1356, 179, 347	BS on the answer + custom comparator + bucket sort - highest-ROI Day 7.
8	Pattern 8 boss + revision drill	LC 148, 215, 973 + full revision	Merge sort linked list + Quickselect bar-raiser. Then the 25-min drill.

Total: ~38 anchor problems across 8 days (the table above is the spine). The full sheet has ~59 problems - once the day's anchors are solid, extend with the rest of each pattern's practice rows. Easy = warm-up; Medium = canonical; Hard = boss. If a problem takes > 30 minutes without progress, peek at the editorial, understand it, write the code from scratch the next day.

The big idea

Arrays look like 50 different problems. They're 9 patterns × 3-9 reps each. The students who get FAANG offers don't grind 200 array problems - they own these 9 patterns cold, and recognize which one fits in 10 seconds.

Do the ~59 must-do problems. Internalize the invariants. Then every future array problem is just a remix.

Pseudocode-first reading: every code block is shown in Python — read each as pseudocode and translate to your interview language (Java / C++ / TS / Go). Single-language keeps each sheet lean; the patterns travel.